

Tiny Village

One-page summary generated from repo evidence only

Evidence used: README.md, main.py, docs/guides/QUICKSTART.md, design_docs/*.md, and core runtime modules.

What It Is

Tiny Village is a Python 2D simulation game where autonomous villagers go about daily life in a dynamic world. Its decision loop combines GOAP planning, utility scoring, a shared graph world model, memory, and optional local LLM routing to turn character state into actions.

Who It's For

Primary user/persona: **Not found in repo.** Current docs mostly target developers, testers, contributors, and first-time users running demos or integration checks.

What It Does

- Runs visual, minimal-console, and test modes from one entry point.
- Simulates autonomous characters with needs, jobs, relationships, inventory, and locations.
- Chooses actions with GOAP, utility scoring, and optional LLM decision support.
- Maintains a shared graph of characters, places, items, events, and relationships.
- Handles scheduled or recurring events and can turn them into story arcs.
- Stores and retrieves character memories, with fallbacks when optional dependencies are missing.
- Provides a pygame map/UI with minimap, stats, weather, and save/load controls.

How It Works

- `main.py` parses CLI options, checks core dependencies, and launches visual, minimal, or test mode.
- `GameplayController` runs the main loop and coordinates characters, events, actions, analytics, and rendering.
- `GraphManager` is the central world state for entities and relationships; other systems read/update it through shared helpers.
- Main planning path: `EventHandler` -> `StrategyManager` -> `GOAPPlanner/utility` functions.
- Optional LLM path: `PromptBuilder` -> `TinyBrainIO` -> `OutputInterpreter`; interpreted actions flow back to execution.
- Executed actions update graph state, memories, and optional story arcs; `MapController` handles visual map interaction.

How to Run

1. Clone and enter the repo: `git clone https://github.com/dhar174/tiny_village.git` then `cd tiny_village`.
2. Install minimum demo dependencies: `pip install pygame networkx numpy nltk pydantic faiss-cpu scikit-learn pandas`.
3. Fastest start: `python main.py --mode minimal`.
4. Other modes: `python main.py --mode visual` or `python main.py --mode test`.
5. If optional LLM components are unavailable, add `--no-llm`.